Building Machine Learning Pipelines on the Urban Data Integration Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

October 4, 2026

1 Prediction Problem

The selected problem is trip-level taxi fare prediction. The objective is to predict the base fare amount, measured in US dollars, for an individual taxi trip. The target variable is `fare_amount`, making this a supervised regression task.

The prediction uses trip, temporal, spatial, weather, and air-quality context. Pickup and drop-off zones are assumed to be known when the prediction is made. Trip distance is treated as an estimated route distance available before the trip, although the dataset contains the final recorded distance.

2 Training Dataset Design

Task 1 constructs the modelling dataset from the integrated taxi-trip Delta table. The target variable is `fare_amount`, renamed to `label`. Rows with missing or negative fare amounts or trip distances are removed.

The dataset includes trips whose pickup borough is Bronx, Brooklyn, Manhattan, or Queens. These boroughs each contain more than 10,000 trips in the Train partition. Staten Island is excluded because it does not meet this threshold.

A chronological split is used to preserve the temporal ordering of the data. January and February form the Train partition, while March is divided equally into Validation and Test partitions.

Table 1: Chronological dataset split.

Partition	Date range	Number of rows
Train	1 January–1 March 2024	5,870,362
Validation	1 March–16 March 2024	1,726,201
Test	16 March–1 April 2024	1,783,894

The selected predictors describe temporal patterns, pickup and drop-off locations, trip characteristics, weather, and air quality. They include trip distance, passenger count, vendor and rate codes, location identifiers, boroughs, temperature, humidity, precipitation, wind speed, pressure, weather condition, and PM2.5 concentration. The weather observations are recorded in UTC but are joined as New York time, so each trip receives weather from 4–5 hours earlier; this is discussed in the evaluation report.

Calendar and cyclical features are derived from the pickup timestamp, including hour, day of week, month, weekend status, and sine/cosine encodings of hour and day of week. The raw timestamp is retained for splitting and auditing but is not included directly in the feature vector. Monetary columns such as tips, tolls, taxes, and total amount are excluded because they would introduce target leakage.

The resulting dataset is saved as a Delta table partitioned by `split`, allowing the three partitions to be loaded independently by the downstream tasks.

3 Feature Engineering Strategy

Task 2 fits a Spark feature-engineering pipeline using only the Train partition. The fitted pipeline is then applied unchanged to Validation and Test to prevent information leakage.

Missing numerical values are replaced using medians calculated from Train. Snow depth is excluded because it is entirely missing in the training data. Missing categorical values are represented by an `Unknown` category. Categorical variables are indexed and one-hot encoded, with unseen values handled by an additional category.

Continuous predictors are transformed with `RobustScaler`, whose quartile-based scale is not distorted by extreme values. Centering is disabled to preserve sparse feature vectors. Binary, cyclical, and one-hot encoded features are not scaled, and the target variable is left in its original dollar unit.

The transformed columns are assembled into a final 561-dimensional feature vector. Task 2 saves both the transformed Delta dataset and the fitted feature `PipelineModel`. These artifacts are reused directly by Task 3 for model training, evaluation, and construction of the complete prediction pipeline.

The pipeline stages are:

$$\texttt{SQLTransformer} \rightarrow \texttt{Imputer} \rightarrow \texttt{StringIndexer} \rightarrow \texttt{OneHotEncoder} \rightarrow \texttt{VectorAssembler} \rightarrow \texttt{RobustScaler} \rightarrow \texttt{VectorAssembler}.$$

4 Machine Learning Pipeline

Task 3 implements the model-training, evaluation, persistence, and retraining stages of the machine-learning workflow. It consumes the Delta feature dataset and fitted Spark feature `PipelineModel` produced by Task 2. The feature dataset contains the trip identifier, pickup timestamp, chronological split, label, and assembled feature vector. Task 3 therefore trains directly on the persisted feature vectors without redefining or refitting feature engineering.

Before training, Task 3 validates the required schema and verifies that the Train, Validation, and Test partitions are present and non-empty. It checks for invalid labels and records the date range and row count of each partition. The split counts are also compared with the upstream Task 1 dataset to prevent a stale Task 2 feature table from being used after the training data has been regenerated.

Table 2 summarises the data received by Task 3. The final feature vector contains 561 dimensions.

Table 2: Task 3 input dataset summary.

Partition	Number of rows
Train	5,870,362
Validation	1,726,201
Test	1,783,894
Feature-vector dimension	561

A regularised Spark `LinearRegression` estimator is fitted using the configuration shown in Table 3. The estimator does not apply additional standardisation because the continuous inputs have already been transformed by the fitted feature pipeline.

Table 3: Linear regression configuration.

Parameter	Value	Purpose
regParam	0.1	Applies regularisation to reduce sensitivity to noisy or correlated features.
elasticNetParam	0.0	Uses L2 regularisation without an L1 component.
maxIter	50	Limits the maximum number of optimisation iterations.
solver	L-BFGS	Provides scalable optimisation for the linear regression model.
standardization	False	Avoids repeating the scaling already performed by Task 2.

The estimator is fitted only on the Train partition. The trained regression model is evaluated on Validation and Test without refitting. Task 3 computes mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2). A predictor that always returns the Train mean fare is evaluated as a baseline.

To reduce memory pressure, Task 3 does not cache the complete transformed Validation and Test datasets. The function `evaluate_split()` computes the model errors, baseline errors, label sum, and label squared sum in one Spark aggregation per split.

Table 4 presents the observed results.

Table 4: Task 3 fare prediction results.

Model	Split	MAE ($)	RMSE ($)	R^2
Linear regression	Validation	6.81	10.55	0.612
Linear regression	Test	7.00	10.81	0.618
Train-mean baseline	Validation	10.90	16.94	-0.001
Train-mean baseline	Test	11.19	17.51	-0.002

On the Test partition, the trained model reduces MAE by approximately 37.5% and RMSE by approximately 38.3% compared with the Train-mean baseline. The model's MAE increases by only approximately 2.8% from Validation to Test, while Test R^2 remains similar to Validation R^2. These results indicate stable generalisation without clear evidence of severe overfitting.

After fitting, Task 3 combines the loaded feature-transformer stages with the new regression model:

`Task 2 Transformer Stages` → `LinearRegressionModel`.

The combined model is saved as one complete Spark `PipelineModel` at:

`data/models/fare_prediction_pipeline`.

The saved model accepts raw rows compatible with the upstream training schema, performs preprocessing, and produces fare predictions. Persistence is verified by reloading the model and applying it to 100 raw test rows. The notebook asserts that all resulting predictions are non-null and non-NaN. Task 3 also writes `artifacts/week4_fare_metrics.json`. The manifest records the run identifier, upstream Delta versions, feature pipeline path, vector dimension, model parameters, Spark configuration, split profiles, metrics, pipeline stages, and execution times.

The measured Task 3 execution time was approximately 100.4 seconds on a Windows laptop with the configuration in Section 6.3.3 (4 GB driver, two threads). Model fitting required 88.2 seconds, evaluation 3.1 seconds, and model persistence 7.7 seconds. With the same configuration on a MacBook Pro (Apple M5 Pro, 48 GB RAM), model fitting took 27.1 seconds (median of three runs; see the evaluation report).

5 Engineering Decisions

5.1 Chronological Splitting and Leakage Prevention

Task 1 uses fixed chronological partitions rather than a random split. Borough selection is calculated from Train only, and all learned feature preprocessing is fitted exclusively on Train. Validation and Test therefore represent future periods that do not influence population selection, imputation, category mapping, or scaling.

5.2 Robust and Reusable Feature Processing

Task 2 stores feature engineering as a fitted Spark `PipelineModel`. This makes preprocessing identical during training, evaluation, persistence testing, and retraining. Robust scaling is applied only to continuous variables; binary, cyclical, and one-hot encoded values retain their original ranges.

5.3 Machine Learning

5.3.1 Reuse of Feature Artifacts

Task 3 directly loads the feature Delta table and fitted feature `PipelineModel` produced by Task 2. It does not redefine feature selection or preprocessing. The shared feature definition is therefore the single source of truth for the feature-vector structure, and no feature logic is duplicated. The feature definitions are stored in `src/data_intensive_computing/week4/features.py`, and the regressor and `evaluate_split()` in `modeling.py`. Task 2, Task 3, and both Task 4 workflows import these modules.

Before training, Task 3 validates the required columns, chronological splits, label values, feature dimension, and split counts. The split counts are compared with the upstream training table to detect stale Task 2 output.

5.3.2 Training and Evaluation

Regularised Spark `LinearRegression` was selected because the assignment focuses on a reproducible workflow rather than maximum accuracy. The model is trained only on the Train split and evaluated on Validation and Test using MAE, RMSE, and R^2. A Train-mean predictor is included as a baseline.

To reduce local memory usage, Validation and Test predictions are not cached. All model and baseline metrics for one split are calculated in a single Spark aggregation.

5.3.3 Persistence and Retraining

The fitted regression model is combined with the transformer stages loaded from Task 2 and saved as one complete `PipelineModel`. The saved model is reloaded and tested on raw rows to verify that preprocessing and prediction can be reproduced together.

Retraining is implemented as a batch workflow. After new data becomes available, Tasks 1, 2, and 3 are rerun in order. Task 3 records the upstream Delta versions, model parameters, metrics, Spark configuration, and execution times in a JSON manifest.

6 Trade Offs

6.1 Feature Coverage versus Contextual Information

Weather variables have broad temporal coverage, while borough-hour PM2.5 is sparse. Removing air quality would simplify the pipeline but discard available urban context. The workflow therefore retains mean PM2.5, applies Train-only median imputation, and includes a missingness indicator. In practice, only the indicator reaches the model: after imputation, so many rows share the median that the interquartile range of PM2.5 is zero, and `RobustScaler` maps every value to zero. Passenger

count and precipitation are affected in the same way. Snow depth is removed because it is completely missing in Train and therefore cannot be imputed reliably.

6.2 Robust Scaling versus Outlier Removal

The source data contains extreme fare and distance observations, including trips longer than 10,000 miles. Automatically removing these records would require an additional domain-validity assumption, so Task 2 keeps them. `RobustScaler` divides each continuous value by its Train interquartile range. This keeps the scale itself stable, because quartiles are not affected by extreme values, but it does not reduce their influence: an extreme distance remains extreme after scaling and still dominates the squared-error fit. The trained model gives `trip_distance` a coefficient close to zero, which is most likely caused by these outliers. Explicit outlier filtering or a robust loss such as Huber loss, which Spark supports, would be the next step.

6.3 Machine Learning

6.3.1 Simplicity versus Predictive Performance

Linear regression is scalable, reproducible, and inexpensive to train, but it cannot represent complex non-linear interactions as effectively as tree-based models. This trade-off is acceptable because the objective is to demonstrate the ML workflow rather than maximise prediction accuracy.

6.3.2 Artifact Reuse versus Pipeline Dependency

Reusing Task 2 artifacts prevents duplicated feature-engineering code. However, it creates a strict execution dependency: Task 1 $\rightarrow$ Task 2 $\rightarrow$ Task 3. Task 3 therefore validates split counts to detect stale feature data.

6.3.3 Memory Usage versus Runtime

The local Spark configuration uses a 4 GB driver and two execution threads, and the evaluation datasets are not cached. This reduces the risk of JVM and Windows page-file failure, although it may require some data to be read more than once.

6.3.4 Batch Retraining versus Incremental Learning

Spark MLlib linear regression does not support incremental `partial_fit`. The current workflow therefore retrains the model on the complete current Train split. This is more expensive than an online update but ensures that the model is consistent with the latest feature pipeline and training data.